THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 1
How Programs Run: Compilers, Interpreters, and JIT Compilation (a Java Case Study); the Language-Processing System; Why Study Compilers; Kinds of Compilers

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.1 (Language Processors), §1.2 (Language-Processing System – previewed here, detailed in Lecture 2), §1.4 (Compiler/Interpreter Distinction), §1.5 (Applications of Compiler Technology)
Other references:	[ET] Engineering a Compiler §1.1; [AP] Modern Compiler Implementation, Ch. 1

Here is what today actually covers:

Lecture Roadmap

1. What is a language processor, and what exactly is a *compiler*?
2. Compilers vs. interpreters vs. just-in-time (JIT) compilation — three ways to run a program, with a full case study of how **Java** uses this to become portable.
3. A **typical language-processing system**: preprocessor, compiler, assembler, linker, and loader – and exactly what an **object file** is.
4. Why study compilers at all, even if you never write one professionally?
5. Applications of compiler technology far beyond "turning C into an executable."
6. Six different **kinds of compiler** you'll encounter by name in industry.

This lecture is entirely conceptual — no algorithms yet. Lecture **??** goes much deeper into the internal *structure* (phases) of a compiler.

1.1 What Is a Language Processor?

[DB] A **language processor** is any program that takes another program written in some source notation and does something useful with it — runs it, translates it, checks it, or transforms it. Compilers are the most famous member of this family, but not the only one.

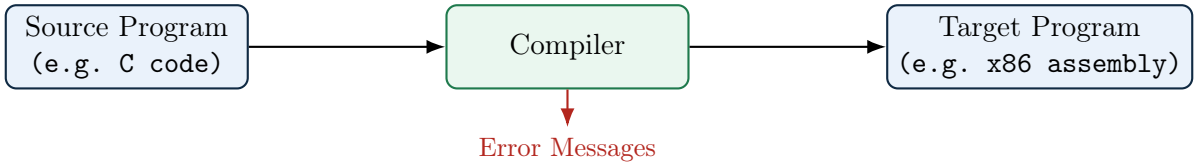
Definition: Compiler

A **compiler** is a program that reads a program written in one language — the *source language* — and translates it into an equivalent program in another language — the *target language*. Along the way, if the source program has errors, the compiler should report them to the user in a useful way.

Three things are packed into that definition and worth pulling apart, because exam questions love to test exactly this distinction:

- **Translation, not execution.** A compiler's job is to *produce* a target program. It does not itself run your code. If you compile `main.c` with GCC, GCC's job ends when it hands you `a.out`; a completely different program (the operating system's loader, then the CPU) is what actually executes it.

- **Equivalence.** The target program must have the same meaning as the source program. This is the single hardest engineering promise a compiler makes, and essentially the entire field of compiler optimization exists to preserve this promise while still making the target program faster.
- **The target language need not be machine code.** Nothing in the definition says what the target language *is* – only that it is "another language." Machine code is the single most common and most visible target, which is why people default to thinking "compiler = produces machine code," but it is only one case. A compiler whose target is *another high-level language* is a source-to-source compiler (§1.6); a compiler whose target is bytecode for a virtual machine (`javac` → JVM bytecode, §1.2.5) is still a compiler by this same definition, even though no physical CPU can execute its output directly.



Exam Focus

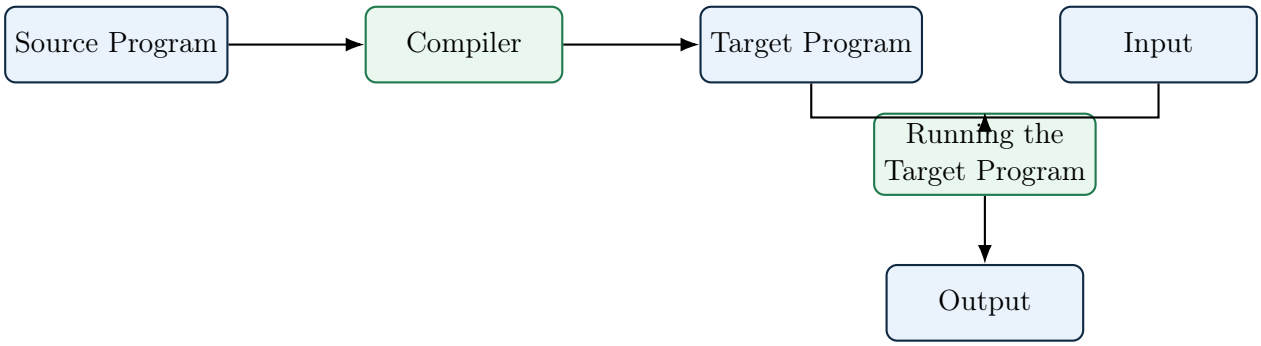
Be able to state, in one sentence, that a compiler *translates*; it does not *execute*. A one-line definition question like "define a compiler" is a classic first-quiz question and students routinely lose marks by describing what an interpreter does instead – or by adding "... into machine code" to the definition, which is not required and is contradicted by transpilers and by `javac` (§1.6).

1.2 Compilers vs. Interpreters vs. JIT Compilation

This is the heart of today's lecture (DB §1.4). There are three broad strategies for getting from source code to a running program, and almost every real language-processing system you use day to day is one of these three, or (very commonly) a hybrid.

1.2.1 Pure Compilation (Ahead-of-Time)

[DB] The compiler translates the entire source program into a target program *before* execution begins, and the target program can then be run any number of times, independently of the compiler. In the classic AOT case that gives this strategy its name, the target language is machine code for the host machine – this is what C, C++, Rust, and (classic) Fortran compilers produce – but the "ahead-of-time, whole-program-at-once" strategy itself is independent of that choice of target (a transpiler, §1.6, is also AOT; it just targets another high-level language instead of machine code).



Advantage – maximal optimization opportunity. Because the entire source program is available before a single instruction runs, an AOT compiler can afford analyses and transformations that a system under run-time time pressure cannot: multiple, deliberately ordered optimization

passes; *whole-program* (interprocedural) analysis such as global register allocation or cross-function inlining; and expensive, exhaustive search over code-generation choices. None of this has to finish before the user sees output, since compilation happens entirely offline – only the resulting target program's run time is on the user's clock.

Disadvantage – the output is not portable. The target program an AOT compiler produces is machine code for one specific instruction set, calling convention, and (usually) operating system. A binary built for x86-64 Linux will not run on ARM64 macOS: there is no platform-independent artifact to ship. Reaching a new target means recompiling from source for that target (or cross-compiling, §1.6) – unlike an interpreter, which ships the same source to every platform, or a bytecode/JIT system, which ships the same portable bytecode everywhere.

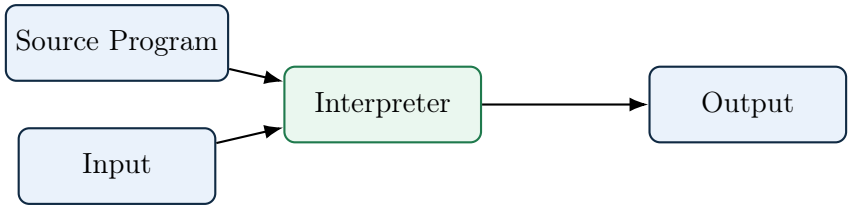
Trade-off, restated: compilation itself can be slow and happens "offline," but every run of the target program afterward is fast, because all the translation work – and all the optimization it enables – has already been paid for once, at the cost of tying the result to one platform.

1.2.2 Pure Interpretation

[DB] An **interpreter** does not produce a separate target program at all. Instead, it directly executes the operations specified in the source program, using the source program (or a close representation of it) as input on every single run.

Definition: Interpreter

An interpreter is a language processor that reads a source program and, using the input supplied by the user, *directly* produces the output of that program — with no separate translation phase that produces standalone target code the user can run independently.



Classic (older) Python, shell scripts, and simple Basic interpreters are close to this pure model: the interpreter re-examines and re-dispatches on the source (or its parsed form) every single time a statement executes, even inside a loop that runs a million times.

Advantage – portability. The same source program (or bytecode) runs unmodified on any machine that has a compatible interpreter, with no separate build step per target platform. This is the mirror image of the AOT compiler's disadvantage above.

Disadvantage – repeated-work overhead. There is no separate compile step, so you can start running – and get feedback on – code instantly, which is wonderful for interactive development and REPLs. The cost is that the interpreter re-examines and re-dispatches on the source (or its parsed form) on *every* execution of a line, with no persistent, whole-program analysis carried over between runs – the opposite of an AOT compiler's one-time, multi-pass optimization. DB notes this repeated-work overhead can make interpretation **10–100× slower** than running compiled code.

Exam Focus

Know this speed trade-off in both directions: compilation trades slower "build time" for faster "run time"; interpretation trades away build time for slower, but more flexible and immediate, run time. Two further pairings are just as commonly tested: *portability* is a classic interpreter advantage over AOT compilation (the same source runs unmodified on any machine that has the interpreter, whereas an AOT binary is tied to one target and needs a recompile elsewhere), while *optimization opportunity* runs the other way – an AOT compiler sees the whole program before

run time and can afford multiple, expensive, whole-program passes, while a pure interpreter re-examines the source on every execution and so cannot afford to.

1.2.3 The Hybrid Reality: Bytecode + JIT

[beyond DB] In practice, almost no serious modern language is *purely* compiled or *purely* interpreted. DB §1.4 briefly mentions Java's model as a hybrid; it is worth understanding this hybrid model properly, because it is how the overwhelming majority of code you run today — Java, C#, JavaScript, modern Python (via PyPy), and even Python's own CPython — actually executes.

Definition: Virtual Machine (VM), vs. a Plain Interpreter

A **virtual machine**, in the sense used here, is a program that simulates a computer: it defines its own instruction set (an abstract, software-only ISA) together with the state a real CPU would have – a program counter, and either an operand stack (JVM, Wasm) or a set of virtual registers – and then runs a fetch-decode-execute loop over that instruction set, exactly the way real hardware executes machine code, except every step is simulated in software. Crucially, a VM does *not* interpret your source program directly. There is always a separate front-end compilation step first (described just below) that translates source into **bytecode** – instructions for this abstract machine – and the VM interprets *that*, not your original code.
This is different from a "plain" (tree-walking) interpreter, of the kind §1.2.2 first introduced: a tree-walking interpreter parses source into an AST and directly, recursively evaluates that tree (`evaluate(node)` calling `evaluate(node.left)`, and so on). There is no separate instruction set, no fixed opcode encoding, and no simulated CPU state – the evaluator's own call stack mirrors the shape of the *source program*, not the shape of a machine. Nothing about it resembles hardware, so it does not earn the name "virtual machine."
The distinction, side by side:

- **What is executed.** VM: a fixed, formally-defined bytecode instruction set, produced by a separate compile step. Tree-walking interpreter: the source program's own parse tree, walked directly, with no separate instruction set at all.
- **Execution state.** VM: an explicit program counter plus an operand stack or virtual registers – state modeled on real hardware. Tree-walker: just the recursive call stack of the `evaluate` function, shaped like the program's grammar, not like a CPU.
- **Is it a genuine compilation target?** VM: yes – bytecode is often documented and shared across multiple front-end languages (JVM bytecode is targeted by Java, Kotlin, Scala, and Clojure alike). Tree-walker: no – the AST is just this one interpreter's private internal representation of this one program.

Every VM is (or contains) some form of interpreter – it interprets bytecode instruction by instruction. But not every interpreter is a VM: a tree-walker interprets source structure directly and never defines a machine to simulate. (A second, unrelated sense of "virtual machine" – *system* VMs like VMware or QEMU, which virtualize an entire computer including its OS – is a different concept entirely and not what is meant here.)

1. **Source → bytecode.** A "front-end" compiler translates source code into a compact, machine-independent intermediate form called **bytecode** (Java's `.class` files hold JVM bytecode; Python's `.pyc` files hold Python bytecode).
2. **Bytecode → execution.** A **virtual machine (VM)** then runs this bytecode. It can do this two ways:
   - *Interpret* the bytecode directly, instruction by instruction (slow but starts instantly) – this is what most bytecode VMs do at first.
   - *Just-in-time (JIT) compile* hot pieces of bytecode — the loops and functions that run

over and over — into native machine code *while the program is running*, and cache that native code for reuse.

Definition: Just-In-Time (JIT) Compilation

JIT compilation defers translation to native machine code until run time, and does so selectively: only code that is actually hot (executed frequently) is compiled, while cold code may remain interpreted. This lets a system get the fast startup of an interpreter and the fast steady-state speed of a compiler.

Why "Warm-Up" Adds a Delay

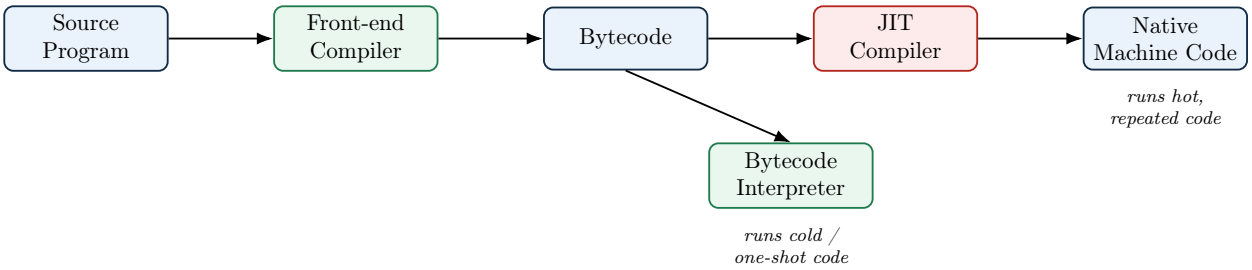
The "time to reach peak speed" row in §1.2.4's table (distinct from startup latency – see the exam-focus note there) has a concrete cause: three costs stack up, in sequence, before any piece of code reaches its fastest version, and none of them can be skipped.

1. **Early runs pay interpreter-speed cost.** The very first time a method or loop executes, no compiled native code for it exists yet – there is nothing to jump to – so it runs through the interpreter (or a cheap baseline tier, e.g. HotSpot's C1), which is inherently slower per operation than optimized native code.
2. **The system must first observe that code is hot.** A JIT does not compile on the first call; it counts. The runtime increments an invocation/back-edge counter on every call or loop iteration and only triggers compilation once that counter crosses a threshold. A loop that only runs a handful of times may never get compiled at all, because counting to that threshold, by construction, requires real executions to have already happened at the slower tier.
3. **Compilation itself takes real, non-trivial time.** Once flagged hot, the method still has to go through the JIT's own pipeline – build an IR from the bytecode, run optimization passes (inlining, register allocation, and more), and emit native code. This usually runs on a background thread so it does not block the program outright, but it still competes for CPU and memory (see "Extra runtime overhead" in §1.2.4's table), and the *old*, slower version keeps running until the newly compiled version is ready and swapped in.

The practical consequence: long-running processes (servers, long batch jobs) amortize this one-time warm-up cost over a long steady-state run and benefit enormously from JIT compilation, while short-lived scripts or CLI invocations may finish before ever reaching peak speed – exactly why AOT-vs.-JIT benchmarks can be misleading if they measure only a program that runs for a second or two.

Example: Tiered execution in real systems

- **JVM (HotSpot):** interprets bytecode first; the C1 ("client") JIT compiles warm methods quickly with light optimization; the C2 ("server") JIT recompiles the hottest methods with heavy optimization once enough profiling data exists.
- **V8 (Chrome/Node.js JavaScript engine):** `Ignition` interprets bytecode; `Sparkplug` does a fast baseline JIT; `TurboFan` does an aggressive optimizing JIT for hot functions, and can *deoptimize* back to the interpreter if an assumption used for optimization (e.g. "this variable is always a number") turns out to be wrong.
- **CPython (standard Python):** traditionally a pure bytecode interpreter with no JIT; `PyPy` is an alternative Python implementation that adds a JIT and is often 5–10× faster on long-running code as a result.



Extra Depth: AOT with a JIT Fallback

DB draws the compiler/interpreter line fairly simply. In practice, the line is blurrier still: some modern systems (GraalVM Native Image, .NET's ReadyToRun) precompile most code AOT for fast startup but still keep a JIT available for code discovered or specialized at run time. So "compiled" and "JIT-compiled" are not always mutually exclusive labels for one system. *(Transpilers, cross-compilers, bootstrapping compilers, and decompilers are their own category of "kind of compiler" rather than a compiler/interpreter distinction – we cover all of these properly in §1.6.)*

1.2.4 Comparing the Three Models

Table 1: Comparing the three execution models

	Pure Compiler	**Pure Interpreter**	**JIT / Hybrid**
When translation happens	Entirely before run time	Continuously, during run time	Source → bytecode happens before run time; bytecode → native code (the JIT step itself) is **always fully during** run time – see note below
Time to reach peak speed	N/A – already at peak from instruction one (see above)	N/A – there is no "peak"; every instruction runs at the same interpreted speed forever	**Nonzero ("warm-up")** – profiling must first identify hot code, then compilation must finish, before that code is upgraded to native speed; the program is already running the whole time this happens
Steady-state speed	Fastest	Slowest (re-dispatch overhead)	Approaches compiled speed for hot code
Portability	Target is machine-specific – a new platform needs a recompile (or cross-compile)	Source runs anywhere the interpreter exists	Bytecode is portable; the JIT-generated native code is not, but it never has to be shipped
Optimization opportunity	Largest – whole program visible before run time, so multiple, expensive, whole-program passes are affordable	Smallest – source (or bytecode) is re-examined on every execution, so heavyweight analysis is normally too costly to repeat	Bounded by run-time budget – fewer/cheaper passes than AOT and limited mostly to hot methods, but compensated by real profiling data (next row) unavailable to AOT

continued on next page

Table 1 – comparing the three execution models (continued)

	Pure Compiler	Pure Interpreter	JIT / Hybrid
Can use runtime profiling info?	No – only sees the source, never the running program	N/A – re-examines source every time, no persistent profile	**Yes** – observes actual branch frequencies and value types, enabling speculative optimizations no static compiler can safely make
Extra runtime overhead	None – compiler is long gone by run time	Dispatch overhead only	Profiling counters, background compiler threads, and a generated-code cache all compete with the program for CPU/memory
Debuggability / interactivity	Weakest (must recompile to test a change)	Best (REPL, immediate feedback)	Good (often still supports a REPL over bytecode)
Typical users	C, C++, Rust, Fortran	Shell scripts, classic BASIC	Java, C#, JavaScript, PyPy

Exam Focus

The "When translation happens" row is the row most often misread. The hybrid pipeline has *two* separate translation stages, and only one of them is "just-in-time":

- **Source → bytecode** (e.g. `javac` producing `.class` files) is ordinary ahead-of-time compilation. It happens once, before the program is ever run, exactly like §1.2.1's pure-compiler case – it is just that its *target* language is bytecode instead of machine code.
- **Bytecode → native code**, the actual JIT step, is *never* done ahead of time, not even partially. The decision of *which* loop or method is hot enough to be worth compiling is made by watching the program actually run (via profiling counters), and the native code generated is specific to the behavior observed *during that run* – this is exactly what "just-in-time" means: translation deferred until the last possible moment, triggered by run-time evidence, not decided in advance.

So the "partly before, partly during" language in the table describes the *full hybrid pipeline* (front-end compile, then JIT), not the JIT step in isolation. Taken alone, JIT compilation is entirely a run-time activity, no different in *when* it happens from pure interpretation – what makes it different is *what* it produces (a reusable native-code artifact) rather than *when* it produces it.

Exam Focus

A related trap, worth knowing even though the table above does not track it separately: "startup latency" and "warm-up delay" sound like the same thing but are not. **Startup latency** asks only: how long before the program starts executing at all? By that measure, JIT and pure interpretation are identical – a JIT-based system *begins* by interpreting (or running a cheap baseline tier) exactly like a pure interpreter does, so there is no extra pause before the first instruction runs. "Warm-up," by contrast, is not a delay *before* starting; it is the time *after* the program has already started before some of its code gets promoted to native speed – exactly what the "Time to reach peak speed" row above captures. The program is running the entire time warm-up is happening – it is simply running at interpreted (not yet peak) speed for that stretch. So it is correct to say a JIT system has low startup latency *and* a nonzero warm-up period – these are not in tension, because they answer two different questions.

A Bit of History

Interpretation is not a modern shortcut invented for scripting languages — it is *older* than compilation. Early LISP (1958) was interpreted because writing a LISP compiler was itself a research problem; interpreters let you experiment with a language before anyone knew how to compile it well. JIT compilation is also older than most people assume: the term traces to Smalltalk and self-modifying-code systems of the 1980s, and was popularized for the mainstream by Sun's HotSpot JVM in 1999 – it was Java's answer to "we want C-like speed but Smalltalk-like portability and safety."

1.2.5 Case Study: How Java Achieves Portability

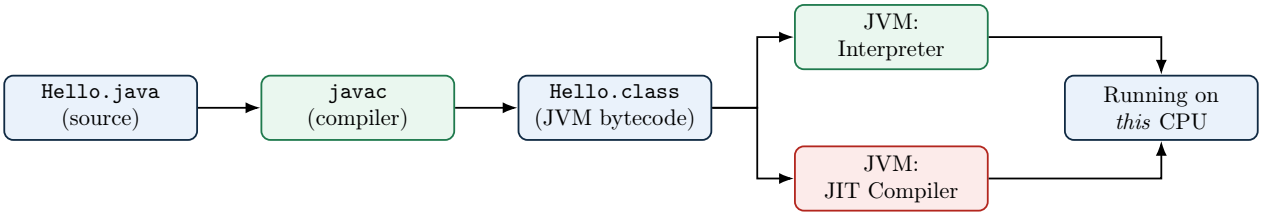
[beyond DB] Java is the single clearest real-world illustration of everything in §1.2, so it is worth walking through as a complete case study — especially since Java's own design goals were *explicitly* about the compiler/interpreter trade-off from the start.

1.2.5.1 From Pure Interpreter to JIT: A Short History

Era	What Happened
Java 1.0 (1996)	The original JVM was a **pure bytecode interpreter**. Every bytecode instruction was fetched, decoded, and dispatched on every single execution – simple and portable, but roughly 10–20× slower than equivalent compiled C, which badly hurt Java's early reputation for performance.
Java 1.2 – "HotSpot" research	Sun's research team built an adaptive compiler codenamed **HotSpot**: instead of compiling everything, or nothing, it would *watch the program run* and compile only the small fraction of methods that actually matter for performance.
Java 1.3 (2000) onward	HotSpot became the default production JVM. This is the point Java became a genuine **hybrid**: start by interpreting (fast startup), then JIT-compile whichever methods turn out to be "hot" (fast steady state). Every mainstream JVM since has kept this same architecture.

1.2.5.2 Compilation to Bytecode, Then Portability via the JVM

[DB] `javac`, the Java *compiler*, does not produce machine code for any real CPU at all. It produces **Java bytecode** – instructions for an abstract, idealized machine called the **Java Virtual Machine (JVM)** that does not physically exist as hardware.



"Write Once, Run Anywhere"

Because `javac` only ever has to target *one* abstract machine (the JVM specification) instead of every physical CPU family, the exact same `Hello.class` file runs unmodified on Windows/x86, Linux/ARM, or a JVM on any other platform — as long as *that* platform has a compliant JVM installed. All the platform-specific work is pushed into the JVM implementation itself (one per platform, built once by the JVM vendor), instead of into every single Java program ever written. This is Sun Microsystems' famous "Write Once, Run Anywhere" pitch for Java, and it is the exact same $M + N$ argument (rather than $M \times N$) you will meet again with LLVM IR in Lecture **??** — just applied to distribution instead of to compiler internals.

1.2.5.3 HotSpot's Tiered JIT in More Depth

[DB] Modern HotSpot uses **tiered compilation**, layering the interpreter and two JIT compilers so the JVM is never stuck doing more work than the code's importance justifies:

1. Every method starts out **interpreted**, while the JVM silently maintains an **invocation counter** (calls) and a **back-edge counter** (loop iterations) for it.
2. Once a counter crosses a threshold, the **C1 (client) compiler** quickly produces a lightly-optimized native version, and also inserts **profiling instrumentation** into it (recording which branches are taken, which concrete types show up at a call site, ...).
3. If the method keeps getting hotter, the **C2 (server) compiler** recompiles it a second time, now using the profiling data gathered by the C1 version to make aggressive, *speculative* optimizations: inlining a virtual method call that has – so far – always resolved to the same concrete type; eliminating a heap allocation entirely if it can prove the object never escapes the method (**escape analysis**); unrolling a hot loop.
4. If a speculative assumption is later violated (a second, different type shows up at that call site), HotSpot **deoptimizes**: it discards the C2 code for that method and falls back to the interpreter (or C1) until it is confident enough to try recompiling again.

Exam Focus

Be able to explain, in your own words, why HotSpot's C2 compiler can safely make optimizations (like assuming a call site's type never changes) that a purely static AOT compiler like GCC never could – and what mechanism (deoptimization) lets it recover if that assumption turns out to be wrong.

1.2.5.4 JIT vs. AOT vs. Interpreter: Advantages and Disadvantages

Pulling together everything from the enhanced table in §1.2.4 and the HotSpot walkthrough above, here is the direct trade-off summary for a JIT compiler specifically:

JIT Advantages	JIT Disadvantages
Can use **real runtime profile data** (actual types, actual branch frequencies) to make optimizations no static compiler can safely attempt – generally impossible for an AOT compiler, which only ever sees the source.	**Warm-up latency**: peak performance is only reached after enough executions to get hot code recompiled – a real problem for short-lived programs (e.g. CLI tools, serverless functions).
Ships one **portable** bytecode artifact instead of a separate binary per target CPU – matching an interpreter's portability, unlike AOT.	Consumes **extra CPU and memory at run time** for profiling counters, the JIT compiler threads themselves, and a generated-code cache – resources an AOT binary or a plain interpreter never spend.
Reaches **steady-state speed close to AOT-compiled native code** for long-running, hot-path-dominated programs – far faster than a pure interpreter ever gets.	**Less predictable** performance moment-to-moment (a method might pause execution while it gets recompiled), which matters for hard real-time systems that AOT compilation suits better.
Can **adapt to the actual host CPU** at run time (e.g. use available vector instructions) without shipping separate binaries per microarchitecture, unlike a single AOT build.	**Significantly more implementation complexity**: an interpreter, multiple JIT tiers, profiling instrumentation, and deoptimization machinery, versus AOT's comparatively simple "compile once, done."

Extra Depth: The Java Class File, Briefly

A `.class` file is itself a small, well-defined binary format worth knowing at a glance: it opens with a **magic number** (`0xCAFEBABE`, a Sun Microsystems in-joke), a class-file version number, a **constant pool** (a table holding every literal, class name, and method/field reference the class uses – similar in spirit to the symbol table we will meet in Lecture **??**), and then the actual bytecode for each method. Tools like `javap -c Hello.class` let you disassemble a compiled class and read its raw JVM bytecode instructions directly – a nice hands-on way to see §1.2.3's "token stream → bytecode" pipeline made completely concrete.

1.3 Why Study Compilers? (DB §1.1)

[DB] You may never write a production C compiler. So why is this course core, and why does almost every strong CS program require it? Three overlapping reasons:

1. **Compilers are, in DB's phrase, "one of the largest and most complex software systems" commonly built** – yet a working one is buildable in a single semester project. Nowhere else in the curriculum will you build something this large, this precisely specified, end to end, from a formal problem statement to a working artifact.
2. **The techniques generalize far beyond compilers.** Finite automata power text editors, network protocol parsers, and input validation. Context-free grammars appear in JSON parsers, config-file readers, and query languages. Data-flow analysis underlies static analyzers, IDE "find usages" features, and security vulnerability scanners. You are not just learning to build a compiler; you are learning the standard toolkit for building *any* system that must process structured, language-like input reliably.
3. **Compilers sit at the hardware/software boundary.** Understanding how a `for` loop becomes register moves and branch instructions demystifies performance – why one piece of "equivalent" code runs 10× faster than another, why cache-friendly code matters, and why some

languages can be faster than others for reasons that have nothing to do with the programmer's skill.

Extra Depth: A "Meta" Argument

There's a fourth reason DB does not spell out explicitly but that experienced engineers often cite: compiler construction is one of the few courses where you are forced to build a *multi-stage pipeline* with cleanly separated concerns (lexing, parsing, semantic analysis, code generation) and *formally specified* correctness at each stage. This is directly transferable to designing any large pipeline system – data pipelines, network stacks, even ML training pipelines – where the same discipline of "define an intermediate representation, verify it, transform it, repeat" shows up again and again.

1.4 Applications of Compiler Technology (DB §1.5)

[DB] DB is explicit that compiler techniques reach "far beyond" translating a general-purpose programming language into machine code. Five application areas DB highlights:

1. **Implementation of high-level programming languages.** The obvious case, but note the historical point DB makes: higher-level languages raised the level of abstraction precisely *because* compiler technology matured enough to make that abstraction affordable in performance terms.
2. **Optimizations for computer architectures.** As hardware has grown more parallel (multicore, SIMD/vector units, deep pipelines, GPUs), the compiler has taken on more responsibility for finding parallelism and hiding memory latency that programmers cannot realistically manage by hand.
3. **Design of new computer architectures.** Hardware designers now co-design instruction sets *with* the compiler in mind (RISC philosophy: keep hardware simple, let the compiler do the scheduling), since a feature only helps if a compiler can reliably exploit it.
4. **Program translations.** Beyond source-to-machine-code, DB lists: binary translation (retargeting compiled binaries to a new instruction set, e.g. Apple's Rosetta 2 translating x86 binaries to run on Apple Silicon), hardware synthesis (compiling a hardware description language like Verilog into actual circuit layouts), database query interpreters (translating SQL into an execution plan), and compiled simulation (compiling a simulation model, e.g. for chip verification, into fast executable code rather than interpreting it).
5. **Software productivity tools.** Type checking, bounds checking, and static program-analysis tools that catch bugs before run time all reuse the same front-end machinery (lexers, parsers, semantic analyzers) that compilers use.

Extra Depth: Applications DB Does Not Mention

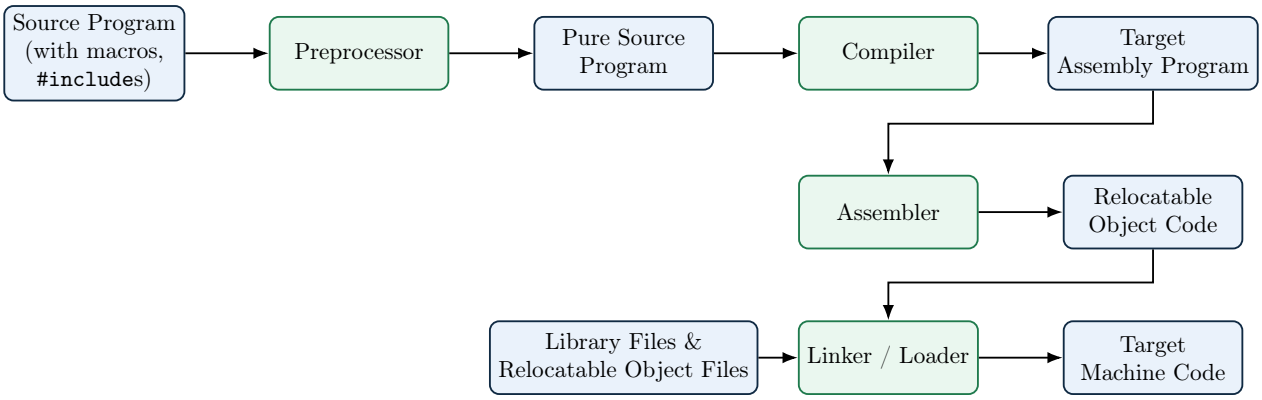
The five areas above were written with early-2000s computing in mind. Several major compiler-technology application areas have grown enormously since and deserve a mention:

- **Machine-learning compilers.** Frameworks like Google's XLA, Apache TVM, and MLIR (Multi-Level Intermediate Representation, itself built using classic compiler theory) compile neural-network computation graphs down to highly optimized GPU/TPU kernels – this is now one of the most active areas of compiler research.
- **WebAssembly (WASM).** A portable, sandboxed bytecode target designed so *any* source language (C, Rust, Go, ...) can compile to it and run safely inside a browser or standalone runtime at near-native speed.

- **Superoptimizers.** Tools (e.g. Souper, STOKE) that search – sometimes with SMT solvers, sometimes with reinforcement learning – for provably optimal short instruction sequences, going beyond what traditional rule-based optimization passes can find.
- **Domain-specific languages (DSLs) and their compilers.** Halide (image-processing pipelines), SQL query planners, regular-expression engines, and shader languages (GLSL/HLSL for GPUs) are all compilers for a language deliberately kept small and specialized so it can be optimized far more aggressively than a general-purpose language could be.
- **Security applications.** Control-flow integrity instrumentation, automatic bounds-check insertion (AddressSanitizer), and fuzzing harness generation are compiler-pass techniques applied specifically to hardening software against attack.
- **LLVM as shared infrastructure.** Perhaps the biggest structural change since DB 2nd edition: LLVM lets dozens of front-ends (Clang/C/C++, Rust, Swift, Julia, and more) share one very well-engineered optimizer and set of back-ends, instead of every language reinventing code generation. We will return to LLVM in Lecture **??**.

1.5 A Typical Language-Processing System

[DB] The "compiler" most people invoke by typing `gcc file.c` is not, strictly, one single program at all – it is a **driver** that coordinates several distinct tools in sequence. DB places the compiler proper inside this larger **language-processing system**.



A typical language-processing system (compare DB Fig. 1.5): the "compiler" you invoke on the command line is really a pipeline of five distinct programs.

- **Preprocessor.** Expands macros (`#define`), includes files (`#include`), and resolves conditional-compilation directives (`#ifdef`) — all as pure text substitution, before the compiler's grammar-aware phases ever run.
- **Compiler (proper).** Everything from §1.2: source → target assembly (today's "black box"; Lecture **??** opens it up phase by phase).
- **Assembler.** Translates human-readable assembly instructions into **relocatable machine code** and packages it as an **object file**.
- **Linker.** Combines multiple object files and library files into one program, resolving references between them.
- **Loader.** Places the final program into memory and starts it running.

Is an Assembler a Compiler?

By the definition in §1.1, yes: an assembler reads a program in one language (assembly) and translates it into an equivalent program in another language (relocatable machine code), with no execution of its own along the way – that is exactly the compiler definition. It is given its own name and its own box in the pipeline diagram above for practical, not definitional, reasons: assembly is designed to be an almost line-for-line, one-to-one notation for machine instructions, so an assembler's job skips most of what makes a "full" compiler hard (there is essentially no analysis phase, no optimization search, and little structural gap between source and target to bridge) and reduces mostly to table lookup and address bookkeeping. The lesson generalizes: *compiler* names a *role* – translate one language into another, without executing either – not a claim about how sophisticated the translation has to be. An assembler, a JVM-bytecode compiler like `javac`, and a heavily-optimizing tool like GCC are all compilers in exactly the same technical sense; they differ only in how much analysis and transformation the translation requires.

Definition: Object File

An **object file** is the output of the assembler: a binary file containing machine instructions and data *that is not yet a runnable program.* Critically, it contains:

- **Relocatable code/data**, with memory addresses left as placeholders, since the assembler does not know where in memory this code will ultimately live;
- A table of **symbols this file defines** (e.g. a function it implements, available for other files to call); and
- A table of **symbols this file references but does not define** (e.g. a call to `printf`, defined elsewhere in the C standard library) – these are exactly what the linker must resolve next.

On Linux this is the `.o` file produced by `gcc -c`; the same idea is called an `.obj` file on Windows.

What the Linker Actually Does

Programs are almost never compiled as a single file. The **linker**'s job is to take *several* object files (and pre-built library files) and merge them into one program, by:

1. **Symbol resolution:** matching every symbol a file *references* (like a call to a function defined in another file, or in a library) to the file that actually *defines* it. An unresolved symbol here is exactly the "undefined reference to `foo`" error you see at link time, as distinct from a compiler error.
2. **Relocation:** now that every piece of code has been assigned a real, final address in the combined program, the linker patches every placeholder address left by the assembler with the correct final value.

The output is one combined file — an **executable** — with (in the simplest case) no unresolved symbols and no remaining placeholder addresses.

What the Loader Actually Does

The **loader** is typically part of the operating system, not the compiler toolchain. When you run a program, the loader:

1. Allocates memory for the program (code, data, stack, heap regions);

2. Copies (or memory-maps) the executable's instructions and initial data into that memory;
3. Performs any *final* address fix-ups that were deferred until this exact moment (relevant for dynamically linked libraries, previewed in Lecture **??**); and
4. Transfers control to the program's entry point – the actual first instruction that runs, which is *not* always your `main` function directly (a small language-runtime startup routine usually runs first).

Are Addresses Absolute After Linking, or Only After Loading?

It depends on which of two linking models is used, and modern systems mostly use the second one.

- **Fixed-base static linking (the "simplest case" above).** The linker assumes the program will always be loaded at one predetermined virtual address. Given that assumption, the linker already has everything it needs to compute true, final **absolute** addresses during linking itself – the "Relocation" step above is the linker permanently writing those absolute values into the executable. The loader's job then is just to copy bytes into memory at that fixed spot; no address math happens at load time.
- **Position-independent executables (PIE) and shared libraries – the modern default.** A shared library (`.so`/`.dll`) must be loadable at different addresses in different processes, since many unrelated programs share one copy of it; and ASLR (address space layout randomization) deliberately loads a program at a different random base address on every run, as a security defense. In both cases the linker *cannot* know the final load address in advance, so it deliberately leaves references as **relative** offsets (or entries in a relocation table) instead of fixing them. Only at load time, once the loader's dynamic-linking component has actually chosen a base address for that run, do those relative offsets become true run-time absolute addresses – this is exactly the "final address fix-ups" mentioned in step 3 above.

So "linker addresses are relative and become absolute once the loader picks where the program is loaded" is correct specifically for the PIE/shared-library case, which is now the common one – not a universal property of everything the linker produces.

Exam Focus

A very common exam trap: "which phase reports an 'undefined reference' / 'unresolved external symbol' error for a missing function?" The answer is the **linker**, not the compiler — the compiler is perfectly happy as long as a function is *declared* (e.g. via a header file); only the linker checks that a matching *definition* actually exists somewhere among all the object files and libraries it was given.

Extra Depth: Seeing the Real Pipeline on Your Own Machine

You can watch this exact pipeline happen with GCC or Clang by stopping it at each stage:

- `gcc -E prog.c -o prog.i` — run *only* the preprocessor; inspect `prog.i` to see macros expanded and headers inlined.
- `gcc -S prog.i -o prog.s` — run the compiler proper; `prog.s` is human-readable target assembly.

- `gcc -c prog.s -o prog.o` — run the assembler; `prog.o` is a relocatable ELF (on Linux) object file. Inspect it with `objdump -d prog.o` or `nm prog.o`.
- `gcc prog.o -o prog` — run the linker, pulling in the C runtime and any libraries, producing the final executable.

Try this on a one-line `"hello world"` program: the jump in size and content between `prog.i` (hundreds of lines, from expanded standard-library headers) and `prog.c` (one line) is a genuinely eye-opening way to internalize what the preprocessor actually does.

Extra Depth: Static vs. Dynamic Linking, and What the Loader Really Does

The linker/loader description above is simplified; two refinements matter a great deal in practice:

- **Static linking** copies the code of every library function you use directly into your executable at link time. The result is self-contained but larger, and does not benefit when the OS wants to share one copy of, say, `libc`, across many running programs.
- **Dynamic linking** instead records, in the executable, only the *names* of needed shared libraries (`.so` on Linux, `.dll` on Windows, `.dylib` on macOS). A specialized program, the **dynamic linker/loader** (`ld.so` on Linux), runs as the executable starts, finds each shared library already resident in memory or maps it fresh, and **patches in** the real addresses of library functions — a process called **relocation** — before your `main` function ever runs. This is why dynamically linked executables are smaller on disk but have slightly higher startup latency and a runtime dependency on the right library versions being present (colloquially, "DLL hell").
- **Position-Independent Code (PIC)** is machine code generated so it can be loaded at *any* memory address, not a fixed one — essential for shared libraries (which may be mapped at different addresses in different processes) and for security features like ASLR (Address Space Layout Randomization). Generating PIC is a back-end code-generation decision, another example of how deep the "compiler" choices reach into topics normally considered OS territory.

Extra Depth: Build Systems Are Compiler Orchestration at Scale

Everything above describes compiling *one* file. Real projects have thousands. The tools that decide *which* files need recompiling and in what order are themselves built on the phase model above:

- **Make** / **Ninja** track file modification timestamps and a dependency graph to avoid recompiling unchanged files — essentially caching the output of the "compiler" node in the pipeline diagram above, keyed by source-file identity and timestamp.
- **ccache** / **sccache** go further: they hash the *preprocessed* source (post-preprocessor, pre-compiler) plus compiler flags, and cache the resulting object file, so an unchanged file compiled with unchanged flags never re-invokes the compiler proper at all — directly exploiting the preprocessor/compiler boundary described earlier in this section.
- **Bazel and other hermetic build systems** extend caching across an entire team or CI fleet: if any engineer, anywhere, has already built a given (source, flags, toolchain) combination, everyone else can fetch the cached object/executable instead of recompiling.
- **Link-Time Optimization (LTO / ThinLTO)** deliberately *breaks* the strict separate-compilation model: instead of each `.c` file being optimized in isolation before linking, LTO defers whole-program optimization to link time, when the optimizer can see every

translation unit at once and, e.g., inline a function across what used to be file boundaries. This directly trades the modularity of the pipeline in §1.5 for better runtime performance.

Real Object-File Formats – Beyond This Introduction

This lecture's treatment of object files, linking, and loading is deliberately introductory. Real object-file formats – ELF on Linux, Mach-O on macOS, PE on Windows – each encode the relocatable code/data, symbol tables, and section layout described above in a fully specified binary format, with their own tools (`readelf`, `otool`, `dumpbin`) for inspecting them. That level of detail is not covered in this course, but is worth exploring on your own with the `objdump`/`readelf` commands mentioned above.

1.6 Different Kinds of Compilers

[beyond DB] "Compiler" is used as an umbrella term across the industry for several genuinely different tools, distinguished by *what kind of language* sits on each side of the translation (high-level vs. low-level) and *which machine* runs the result. Six worth knowing by name:

Kind	Source → Target	Example(s)
Native (AOT) compiler	High-level → machine code, same target as the host machine	GCC, Clang, `rustc` compiling for the machine they run on
Source-to-source compiler (transpiler)	High-level → a *different* high-level language	TypeScript → JavaScript; Babel (modern JS → older JS); CoffeeScript → JavaScript
Cross-compiler	High-level → machine code for a target *different* from the host running the compiler	Compiling ARM binaries for a phone on an x86 laptop; embedded firmware toolchains
Binary translator	Low-level (one instruction set) → low-level (a *different* instruction set)	Apple's Rosetta 2 (x86-64 → ARM64); QEMU's user-mode emulation
Decompiler	Low-level (machine code or bytecode) → high-level-*like* source	Ghidra, IDA Pro (machine code); JD-GUI, CFR (Java bytecode)
Bootstrapping (self-hosting) compiler	A compiler for language X, *written in* X itself	GCC (mostly C); `rustc` (written in Rust)

Example: Two of These, Worked Through

Binary translation (Rosetta 2): when Apple moved Macs from Intel (x86-64) to Apple Silicon (ARM64) in 2020, millions of existing apps were compiled x86-64 binaries with no source available to the end user. Rosetta 2 translates those already-compiled x86-64 binaries into ARM64 machine code – notice both sides of this translation are *low-level*, which is what makes

it a binary translator rather than an ordinary compiler.
Bootstrapping (GCC): you cannot compile the very first C compiler *using* a C compiler, since none exists yet. Historically, an initial, minimal C compiler was written in assembly or an even earlier language; once that minimal compiler could compile *a subset* of C, a fuller C compiler was written in C and compiled by the minimal one; that fuller compiler then compiles *itself* (and every subsequent version) from then on. This chicken-and-egg-breaking process is exactly what "self-hosting" means.

Exam Focus

Given a described tool ("translates Kotlin to Java source," "translates ARM machine code back into readable pseudo-C," "compiles Swift for an iPhone from a Mac"), be able to name which of these six kinds it is and justify your answer using the *source* → *target* column above – this classification-by-example is a very common short-answer format.

Where the Compiler-vs-Interpreter and Kind-of-Compiler Axes Meet

These two classification schemes are independent and can combine freely. A JIT compiler *is* a native compiler by this section's scheme (high-level/bytecode → machine code), just one whose *timing* (during execution) differs from an AOT native compiler's. A transpiler like Babel is almost always itself an AOT tool – it finishes its work entirely before the JavaScript it produces ever runs. Don't confuse "what kind of translation is this" (this section) with "when does the translation happen" (§1.2) – exam questions sometimes test both axes on the same tool deliberately.

1.7 A Preview: What's Next

Lecture **??** opens up the compiler "black box" from §1.2 of this lecture and walks through its internal phases – lexical analysis, parsing, semantic analysis, intermediate code generation, optimization, and final code generation – in real depth, including how the phases group into a front-end, middle-end, and back-end. Everything in today's lecture is about *what* these systems do from the outside; Lecture **??** begins answering *how*, in detail.

1.8 Self-Check Questions

Practice – Not Graded

1. In one sentence, explain why "a compiler executes your program" is a factually incorrect statement, and what does execute it instead.
2. Classic CPython has no JIT; PyPy does. Predict, with a reason, which one would be faster for a program consisting of a single loop that runs 50 million times, versus a script that runs once and exits immediately.
3. Explain, using the $M + N$ vs. $M \times N$ argument, why compiling to JVM bytecode makes Java portable in a way that compiling directly to x86 machine code would not.
4. A program compiles successfully with no errors, but fails at link time with "undefined reference to `compute_total`." Which tool caught this, and why couldn't the compiler have caught it earlier?
5. Is a transpiler (e.g. TypeScript → JavaScript) a compiler under DB's definition? Justify

your answer using the definition given in §1.1, and separately, name which of the six "kinds of compiler" from §1.6 it belongs to.

6. Give one example, not listed in this lecture, of a real tool on your own laptop that is best described as an "interpreter" rather than a compiler.
7. HotSpot's C2 JIT only compiles a method after it has been *interpreted* many times. Why not just JIT-compile every method the very first time it's called?

Key Takeaways

- A **compiler translates**; it does not itself execute the source program. An **interpreter** executes the source directly, with no independent target program left behind.
- Compilers trade slower build time for faster, repeatable run time, at the cost of a target binary that is **not portable** (tied to one ISA/OS) and paid for by the whole-program visibility that enables aggressive, multi-pass optimization. Interpreters trade that away for instant startup, easy interactivity, and **source-level portability**, at the cost of repeated re-interpretation overhead on every run.
- Most real systems today (JVM, V8, .NET, PyPy) are **hybrids**: compile to portable bytecode, interpret it at first, then **JIT-compile** the hot paths to native code at run time – Java's own history (pure interpreter in 1.0, HotSpot's tiered JIT from 1.3 onward) is the clearest real example, and it's what makes "write once, run anywhere" work.
- A JIT's big edge over AOT is **using real runtime profile data** for speculative optimization; its costs are **warm-up latency** and **runtime profiling/compilation overhead** – know this trade-off table cold.
- The compiler proper sits inside a larger **language-processing system**: preprocessor → compiler → assembler → linker → loader. The assembler produces an **object file** (relocatable code plus defined/referenced symbol tables); the **linker** resolves symbols and relocates addresses; the **loader** places the program in memory and starts it.
- "Compiler" covers several genuinely different tools: **native/AOT compilers**, **source-to-source transpilers**, **cross-compilers**, **binary translators**, **decompilers**, and **bootstrapping/self-hosting compilers** – know the source→target shape of each.
- Compiler techniques (automata, grammars, data-flow analysis) show up far outside traditional compilers – in IDEs, static analyzers, database engines, and ML frameworks – which is the real reason this course is core, not elective.
- DB §1.5's five application areas (language implementation, architecture optimization, architecture design, program translation, productivity tools) have since been joined by ML compilation, WebAssembly, and shared infrastructure like LLVM.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.